Patient Flow Simulator

EFG Apps · Test emergency-pathway and SDEC design changes before you make them in the hospital.

Two models in one web app:

- **Whole pathway.** ED → AMU / SDEC → inpatient wards → discharge. It shows where the bottleneck is, how full each unit gets, and what a change does to ED 4-hour and 12-hour performance, ambulance handover, escalation and length of stay.
- **SDEC design.** Whether to co-locate SDEC streams, where to put the unit, how big to make it, what mix of trolleys and chairs, and what hours to open. Includes a bleed check that shows whether sharing space hurts the smaller pathways.

Everything runs in the browser. There is no server, no login and no patient data, and nothing a user types leaves their computer.

What it does

Whole pathway

Demand	Ambulance, ED walk-in, GP direct to AMU, walk-in to other locations. Each source has its own rate, arrival pattern through the day, and acuity (how likely it is to be admitted).
Pathway	Level 1 front door, Level 2 AMU and SDEC, Level 3 wards. Wards can be grouped by division or split by specialty, with presets for a large acute trust and for medium and small district general hospitals.
Discharge	Discharge-time curve (% discharged by midday and by 5pm), weekly pattern (classic weekend dip, flat, or Friday peak), and delays for patients with no criteria to reside, split by pathway P0–P3.
Pressure valves	Outlying to other specialties and escalation beds, each with a length-of-stay penalty, plus repatriation when a home-ward bed frees up.
Outputs	ED 4-hour and 12-hour, ambulance handover, escalation, NCTR census, admitted LOS, ward occupancy, a ranked bottleneck list, a slide-ready flow map and a plain-English summary of each run.

SDEC design

Streams	Medical, surgical, older persons, gynae, or your own, each with its own demand, referral sources, review time, test time (different for admitted and discharged patients), conversion rate, and trolley and chair needs.
Layout	Separate units, one co-located unit, or co-located with each stream's space partly ring-fenced.
Location and hours	Next to ED, next to AMU or remote; opening hours; last acceptance; holding out-of-hours patients until SDEC opens.
Automatic tests	Every run compares the layouts on identical patients, sweeps size from 50% to 150%, tests closing times from 16:00 to 24:00, and runs a surge test on the largest stream.
Show the working	The rules in plain English, a hand calculation alongside the simulation, patient-accounting checks that must balance, and sample patient journeys with clock times.

How to use it well

1. **Calibrate first.** Set the baseline to how your hospital runs today and adjust inputs until ED performance, occupancy and length of stay match your own data. The built-in presets are illustrative starting points, not anyone's real hospital.
2. **Pin a baseline**, change one thing, then re-run. Every result shows the change against the baseline.
3. **Compare, don't forecast.** The model is built to compare options under the same assumptions. A difference between two scenarios is more reliable than any single number.
4. **Check across seeds.** Change the random seed a few times before relying on the direction of a result.
5. **Save and share scenarios** with Export config and Import config.

The technical specification explains every rule, assumption and check in detail, and is the document to hand to an analyst or reviewer.

Running it

Option 1: open the file. Download `patient-flow-simulator.html` from the latest release and double-click it. It works offline.

Option 2: run from source. Needs Node.js 18 or later.

```
npm install
npm run dev            # live preview at http://localhost:5173
npm run build          # dist/ folder for any web host
npm run build:single   # dist-single/index.html, the one-file version
```

Hosting on GitHub Pages

The repository includes a workflow (`.github/workflows/deploy.yml`) that builds and publishes the site on every push to `main` .

1. Push this repository to GitHub.
2. Go to **Settings → Pages** and set **Source** to **GitHub Actions**.
3. Push to `main` , or run the workflow from the **Actions** tab.

The site appears at `https://<your-username>.github.io/<repository-name>/` .

Things to know:

- **Visibility.** GitHub Pages is free for public repositories. Publishing from a private repository needs a paid GitHub plan, and on most plans the published site is public either way.
- **Saved scenarios** are kept in each visitor's own browser storage. They are not shared between users or devices, and clearing browser data deletes them, so export anything important.
- **No tracking or external calls.** Fonts and code are bundled into the site, so it makes no requests to other servers.

For intranet or other hosting, see HOSTING.md.

Project structure

```
src/
  App.jsx          tab bar and EFG Apps branding
  FlowModel.jsx    whole-pathway model: engine and interface
  SdecModel.jsx    SDEC design model: engine and interface
  storage.js       saved scenarios via browser storage
  index.css        fonts and base styles
docs/
  TECHNICAL-SPEC.md
  img/             screenshots used in this README
```

```
.github/workflows/deploy.yml
```

Limitations

Both models are simplifications. The main ones are listed here; the technical specification has the full list.

- Lengths of stay are drawn from statistical distributions, not from your own patient-level data.
- Patients are seen first come, first served, with no clinical prioritisation.
- There is no elective stream competing for beds, no seasonal variation and no staffing constraint beyond the clinician counts in SDEC.
- In SDEC, diagnostic turnaround is a fixed input and does not lengthen when the unit is busy.

Use the outputs to compare options and to show why a change helps or doesn't. For major investment decisions, confirm the shortlisted option with a patient-level model built on your own data.

EFG Apps